

Design a Library Management System

Infosys · TCS · Wipro · Accenture – service MNC interviews lo **అత్యంత సాధారణమైన** LLD problem. అందరూ దీన్ని "CRUD" అనుకుంటారు.

మూడు విరుపులు. మొదటిది ఒక కొత్త ప్రతి కొంటే **ఒకరి loan ని మాయం** చేస్తుంది. రెండోది తిరిగి ఇచ్చిన పుస్తకానికి **జరిమానా పెంచుతూనే** ఉంటుంది. మూడోది **24 రోజులు ఎదురుచూసిన వ్యక్తిని** పక్కన పెడుతుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output నిజంగా `node` `to run` చేసినదే.

ఈ **PROBLEM** ఎందుకు ప్రమాదకరమైనది

ఇది **సులభంగా అనిపిస్తుంది**, మరియు అదే ఉచ్చు. `Book`, `Member`, `Library` – మూడు classes, `borrow`, `return` – ఐదు నిమిషాల్లో పూర్తి.

ఆపై interviewer అడుగుతాడు: "అదే పుస్తకం 5 ప్రతులు ఉంటే?", "ఆలస్య జరిమానా?", "ఎవరైనా ముందే `reserve` చేసుకోవచ్చా?" – మరియు ఆ మూడు classes మూడుసార్లు విరిగిపోతాయి.

ఈ doc ఆ మూడు ప్రశ్నలనీ ముందే అడిగి, విరగొట్టి, సరిచేస్తుంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇక్కడ దాగిన మూడు ప్రశ్నలు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: "పుస్తకం" అంటే ఏమిటి?

3. Step – ఒక `Book` class, ISBN ఒక తాళంచెవి
4. మొదటి విరుపు – కొత్త ప్రతి కొంటే ఒకరి loan మాయమైంది
5. Step – Title మరియు Copy · రెండు వేర్వేరు వస్తువులు

Part 3 – రెండో విరుపు: ఆగని జరిమానా

6. Step – (ఇప్పుడు – గడువు) × రేజీ ₹5
7. రెండో విరుపు – తిరిగి ఇచ్చినా జరిమానా పెరుగుతోంది
8. Step – తిరిగి ఇచ్చిన క్షణంలో గడ్డకట్టడం · grace · cap

Part 4 – మూడో విరుపు: నిరీక్షణ జాబితా

9. Step – "reserve" అంటే పేరు రాసుకోవడం
10. మూడో విరుపు – 24 రోజులు ఎదురుచూసిన ravi ఓడిపోయాడు
11. Step – ప్రతి తిరిగొచ్చినప్పుడు కేటాయించడం

Part 5 – పూర్తి system

12. Step – పునరుద్ధరణ, పరిమితులు, మరియు గడువు మీరిన hold
13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటీతో చెప్పాల్సిన English script
16. Follow-ups – బహుళ శాఖలు, e-books, notifications
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇక్కడ దాగిన మూడు ప్రశ్నలు

"Design a library management system. Members can search for books, borrow them, and return them."

ఒక్క వాక్యం, మూడు classes – అనిపిస్తుంది. కానీ:

మూడు ప్రశ్నలు • మూడు వేర్వేరు నమూనా తప్పులు

"Search, borrow, return" – ప్రతిదాని కింద ఒక ఊహ ఉంది

1 • "పుస్తకం" అంటే? Search చేసేది ఒక శీర్షిక తీసుకునేది ఒక భౌతిక ప్రతి ఇవి ఒకటే అనుకుంటే – → §4 • loan మాయం	2 • జరిమానా "అలస్యమైన రేజులు × రేటు" సరైనదే – ఎప్పుడు లెక్కించాలి? "ఇప్పుడు" అంటే అగడు → §7 • ₹18,250	3 • నిరీక్షణ జాబితా పేర్లు రాసుకోవడం సులభం కానీ ప్రతి తిరిగొచ్చినప్పుడు ఆ జాబితాని ఎవరు చూస్తారు? → §10 • 24 రేజులు వృథా
--	---	--

ఈ మూడింటికీ ఒకే ఆకారం

ప్రతి సందర్భంలోనూ మొదటి పరిష్కారం **పనిచేస్తుంది** – ఒక ప్రతి, ఒక సభ్యుడు, ఒక రేజు.
అది వరిగేది **రెండే ప్రతి** వచ్చినప్పుడు, **రెండే వారం** గడిచినప్పుడు, **రెండే వ్యక్తి** ఎదురుచూసినప్పుడు.
అంటే – **నిజమైన గ్రంథాలయంలో ఎప్పుడూ.**
ఇదే "సులభమైన" problems యొక్క స్వభావం: అవి tests లో కాదు, వాడకంలో ఎరుగుతాయి.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఒక పుస్తకం పలు ప్రతులు ఉంటాయా?	అవును – మరియు ఇదే §4. మొదటి ప్రశ్న ఇదే అడగండి
అలస్య జరిమానా ఉందా? Grace period? గరిష్ట పరిమితి?	§8 – మూడూ వేర్వేరు నియమాలు

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ముందే reserve చేసుకోవచ్చా?	అవును అంటే §11 – ఇది ఒక న్యాయ ప్రశ్న
పునరుద్ధరణ (renew) ఉందా? ఎన్నిసార్లు?	§12 – మరియు వేరేవాళ్ళు ఎదురుచూస్తుంటే?
ఒక సభ్యుడు ఎన్ని పుస్తకాలు తీసుకోవచ్చు?	§12 – ఒక సాధారణ పరిమితి
పుస్తకం పోతే / పాడైతే?	Copy కి ఒక స్థితి కావాలి (§5)
బహుళ శాఖలు?	అవును అంటే Copy కి ఒక శాఖ కూడా (§16)

మొదటి ప్రశ్న అడగకపోతే, మే మొత్తం DESIGN తప్పు పునాది మీద నిలబడుతుంది

"గ్రంథాలయంలో 'Clean Code' యొక్క 5 ప్రతులు ఉన్నాయి. Search చేస్తే ఒక ఫలితం రావాలా, ఐదా?"

జవాబు: ఒకటి – "Clean Code, 5 ప్రతులు, 2 అందుబాటులో".

కానీ borrow చేసేటప్పుడు? – ఒక నిర్దిష్ట ప్రతి బయటికి వెళుతుంది, దానికి ఒక barcode ఉంటుంది, మరియు అది పోవచ్చు, పాడవచ్చు.

అంటే రెండు వేర్వేరు విషయాలు: శీర్షిక(కేటలాగ్‌లో ఉండేది) మరియు ప్రతి(అరలో ఉండేది). ఒక్క Book class రాస్తే – §4 lo ఏమవుతుందో చూద్దాం.

Part 2 – మొదటి విరుపు: "పుస్తకం" అంటే ఏమిటి?

3. Step – ఒక Book class, ISBN ఒక తాళంచెవి

```
class Book {
  constructor(isbn, title, author) {
    Object.assign(this, { isbn, title, author, available: true, borrower: null });
  }
}

class Library {
  constructor() { this.books = new Map(); } // isbn → Book
  add(book) { this.books.set(book.isbn, book); }

  search(q) {
    return [...this.books.values()]
      .filter(b => b.title.toLowerCase().includes(q.toLowerCase()));
  }

  borrow(isbn, member) {
    const b = this.books.get(isbn);
    if (!b) return { ok: false, reason: 'NOT_FOUND' };
    if (!b.available) return { ok: false, reason: `ALREADY_OUT: ${b.borrower}` };
    b.available = false; b.borrower = member;
    return { ok: true };
  }
}
```

ఇది పనిచేస్తుంది:

```
search("clean") : [ 'Clean Code' ]
ravi borrow    : { ok: true }
asha borrow    : { ok: false, reason: 'ALREADY_OUT: ravi' }
```

Ravi తీసుకున్నాడు, asha కి "ravi దగ్గర ఉంది" అని చెప్పింది. సరిగ్గా ఉంది.

ఇప్పుడు గ్రంథాలయం అదే పుస్తకం మరో ప్రతి కొంది.

4. మొదటి విరుపు – కొత్త ప్రతి కొంటే ఒకరి loan మాయమైంది

```
lib.add(new Book('978-01', 'Clean Code', 'Martin')); // రెండో ప్రతి
```

Clean Code ఎవరి దగ్గర : ravi

గ్రంథాలయం అదే పుస్తకం మరో ప్రతి కొంది -

ఇప్పుడు ఎవరి దగ్గర : null ← ravi loan మాయమైంది

మొత్తం entries : 1 (2 ప్రతులు కొన్నా)

asha తీసుకోగలదా : { ok: true }

మొదటి విరుపు - రికార్డు లేకుండా RAVI దగ్గర ఒక పుస్తకం ఉంది

మూడు విషయాలు ఒకేసారి తప్పయ్యాయి:

1 • రెండు ప్రతులు కొన్నా books.size ఇంకా **1**. Map తాళంచెవి ISBN, మరియు రెండు ప్రతులకీ అదే ISBN - కాబట్టి రెండేది మొదటిదాన్ని **తుడిచేసింది**.

2 • ఆ తుడిచేయడంలో **Ravi యొక్క loan పోయింది**. System దృష్టిలో అతను ఏమీ తీసుకోలేదు. అతనికి గుర్తు చేయరు, జరిమానా విధించరు.

3 • ఇప్పుడు asha కూడా తీసుకోగలదు. **ఒక్క రికార్డు, రెండు భౌతిక పుస్తకాలు, ఒక్కరే తెలిసినవారు.**

మౌలిక తప్పు: ISBN అనేది **ఒక శీర్షికకి** గుర్తింపు, **ఒక పుస్తకానికి** కాదు. "Clean Code" అనేది ఒక్కటే; కానీ అరలో ఉన్న **భౌతిక ప్రతులు** ఐదు, మరియు అవి ఒకదానికొకటి **భిన్నమైనవి** - ఒకటి బయట ఉండొచ్చు, ఒకటి పాడవ్వచ్చు, ఒకటి పోవచ్చు.

Deep Dive 04 (BookMyShow) §5 lo ఒక బంధువుని చూశాం - అక్కడ ఒకే **Seat** వస్తువు రెండు shows lo పంచుకోబడింది. అక్కడ సమస్య **ఒక వస్తువు అనేక సందర్భాల్లో**; ఇక్కడ సమస్య **అనేక వస్తువులు ఒకే గుర్తింపుతో**. రెండూ ఒకే మూలం నుంచి: **"ఇది ఏమిటి" మరియు "ఇది ఏది" - వీటిని కలపడం.**

5. Step – Title మరియు Copy • రెండు వేర్వేరు వస్తువులు

ఒక TITLE • అనేక COPIES

కటలాగ్ లో ఒకటి • అరలో ఐదు



Search **Title** ని ఇస్తుంది • Borrow ఒక **Copy** ని తీసుకుంటుంది • నిరీక్షణ జాబితా **Title** కి ఉంటుంది
"అందుబాటులో 2" అనేది నిల్వ చేసిన విలువ కాదు - ప్రతుల స్థితుల నుంచి **తెప్పించినది**.

```
class Title {
  constructor(isbn, name, author, price = 0) {
    Object.assign(this, { isbn, name, author, price });
    this.copies = [];
    // ఈ title యొక్క భౌతిక ప్రతులు
  }
}
```

```

    get total()      { return this.copies.length; }
    get available() { return this.copies.filter(c => c.state === 'ON_SHELF').length; }
  }

  class Copy {
    #state = 'ON_SHELF'; // ON_SHELF · LOANED · LOST · REPAIR
    constructor(barcode, title) { this.barcode = barcode; this.title = title; }
    get state() { return this.#state; }
    setState(s) { this.#state = s; }
  }

```

మూడు వివరాలు:

- **available ఒక getter**, ఒక field కాదు. దాన్ని ఎవరూ "update" చేయరు – అది ఎప్పుడూ ప్రతుల స్థితుల నుంచి లెక్కించబడుతుంది. **నిల్వ చేసిన లెక్క పాతబడుతుంది; లెక్కించిన లెక్క పాతబడదు.**
- **Copy కి ఒక స్థితి ఉంది**, ఒక boolean కాదు. **LOST** మరియు **REPAIR** అనేవి "అందుబాటులో లేదు" కాదు – అవి **వేర్వేరు పరిస్థితులు**, మరియు వాటికి వేర్వేరు చర్యలు ఉంటాయి.
- **barcode ప్రత్యేకమైనది**; **isbn** కాదు. అదే ఆ రెండింటి మధ్య నిజమైన తేడా.

```

search("clean") : [ { name: 'Clean Code', total: 5, available: 5 } ]
2 ఇచ్చాక, 1 పోయాక : [ { name: 'Clean Code', total: 5, available: 2 } ]
ప్రతుల స్థితులు      : 978-01-c1=LOANED 978-01-c2=LOANED 978-01-c3=LOST
                      978-01-c4=ON_SHELF 978-01-c5=ON_SHELF

```

Search ఇప్పటికీ ఒక్క ఫలితం ఇస్తోంది – కానీ ఇప్పుడు అది "5 to 2" అని చెప్తోంది. మరియు ఐదు ప్రతులూ **విడివిడిగా** ఉన్నాయి, ఒక్కొక్కటి తన సొంత కథతో.

Part 3 – రెండో విరుపు: ఆగని జరిమానా

6. Step – (ఇప్పుడు – గడువు) × రోజుకి ₹5

ఆలస్య జరిమానా. నియమం సరళం: ఆలస్యమైన ప్రతి రోజుకి ₹5.

```
class Loan {
  constructor(member, dueAt) {
    Object.assign(this, { member, dueAt, returnedAt: null });
  }
  fine(now) {
    const late = Math.floor((now - this.dueAt) / DAY);
    return Math.max(0, late) * 5; // రోజుకి ₹5
  }
}
```

ఇది సరైన సూత్రం. Ravi 10వ తేదీ గడువు, 15వ తేదీన తిరిగి ఇచ్చాడు – 5 రోజులు ఆలస్యం, ₹25.

7. రెండో విరుపు – తిరిగి ఇచ్చినా జరిమానా పెరుగుతోంది

Ravi 10వ తేదీ due, 15వ తేదీన తిరిగి ఇచ్చాడు – 5 రోజులు ఆలస్యం

2026-01-15 న చూస్తే	→	జరిమానా ₹25
2026-01-20 న చూస్తే	→	జరిమానా ₹50
2026-02-09 న చూస్తే	→	జరిమానా ₹150
2026-07-19 న చూస్తే	→	జరిమానా ₹950

రెండో విరుపు – పుస్తకం అరలో ఉంది, జరిమానా ఇంకా పెరుగుతోంది

Ravi 15వ తేదీన తిరిగి ఇచ్చాడు. పుస్తకం అప్పటినుంచీ అరలోనే ఉంది. అయినా జూలైలో చూస్తే అతను **₹950** బాకీ.

మౌలిక తప్పు: జరిమానా అనేది ఒక గత సంఘటన యొక్క పరిణామం – "అతను 5 రోజులు ఆలస్యంగా ఇచ్చాడు". కానీ మనం దాన్ని ఒక ప్రస్తుత లెక్కగా రాశాం – "ఇప్పటిదాకా ఎన్ని రోజులు?".

Deep Dive 11 §4 lo ఇదే ఆకారం చూశాం: order ఒక live MenuItem ని పట్టుకుంటే, ధర మారినప్పుడు **పాత order మారిపోయింది**. ఇక్కడ loan "ఇప్పుడు" ని పట్టుకుంటోంది, కాబట్టి **సమయం గడిస్తే గతం మారిపోతోంది**.

మరియు రెండో సమస్య – ఆగే చోటు లేదు

మరియు ఎప్పటికీ తిరిగి ఇవ్వని ఒక పుస్తకం:
1 సంవత్సరాల తర్వాత → ₹1,825 (పుస్తకం ధర ₹450)

3 సంవత్సరాల తర్వాత → ₹5,475 (పుస్తకం ధర ₹450)
10 సంవత్సరాల తర్వాత → ₹18,250 (పుస్తకం ధర ₹450)

₹450 పుస్తకానికి ₹18,250 జరిమానా. ఇది అర్థరహితం – పుస్తకం పోతే గ్రంథాలయానికి అయ్యే నష్టం **₹450**, అంతే.

నిజమైన గ్రంథాలయాలు ఈ కారణంగానే **గరిష్ట పరిమితి** పెడతాయి: జరిమానా ఎప్పుడూ పుస్తకం ధరని మించదు. దాన్ని మించిన క్షణం, అది జరిమానా కాదు – **అది పుస్తకం కొనుగోలు.**

8. Step – తిరిగి ఇచ్చిన క్షణంలో గడ్డకట్టడం • grace • cap

రెండు మార్పులు. మొదటిది: **నియమాన్ని ఒక వస్తువుగా చేయడం** –

```
class FinePolicy {
  constructor({ perDay = 5, graceDays = 3, capFraction = 1 } = {}) {
    Object.assign(this, { perDay, graceDays, capFraction });
  }
  amount(dueAt, at, price) {
    const late = Math.floor((at - dueAt) / DAY) - this.graceDays;
    return Math.min(price * this.capFraction, Math.max(0, late) * this.perDay);
  }
}
```

రోజుకి రేటు, grace రోజులు, గరిష్ట పరిమితి – మూడూ ఒక్కో గ్రంథాలయానికి ఒక్కోలా ఉంటాయి. ఇవి **if** కాదు, **configuration**.

రెండోది – **గడ్డకట్టడం**:

```
class Loan {
  #settled = null; // తిరిగి ఇచ్చిన క్షణంలో గడ్డకట్టినది

  // ఇంకా బయట ఉంటే – ఇప్పటిదాకా; తిరిగి ఇచ్చాక – గడ్డకట్టిన విలువ
  fine(now) {
    if (this.#settled !== null) return this.#settled;
    return this.policy.amount(this.dueAt, now, this.copy.title.price);
  }
  settle(at) {
    this.returnedAt = at;
    this.#settled = this.policy.amount(this.dueAt, at, this.copy.title.price);
    return this.#settled; // ← ఇక్కడే గడ్డకట్టడం
  }
}
```

ఒక విలువ, రెండు అర్థాలు - మరియు అదే ఇక్కడి కీలకం

పుస్తకం ఇంకా బయట ఉంటే - జరిమానా అనేది ఒక అంచనా: "ఈరోజు తిరిగి ఇస్తే ఇంత". అది మూరాలి.

పుస్తకం తిరిగి వచ్చాక - జరిమానా అనేది ఒక వాస్తవం: "అతను ఇంత బాకీ పడ్డాడు". అది ఎప్పటికీ మారకూడదు.

ఒకే method, రెండు ప్రవర్తనలు - మరియు వాటిని వేరుచేసేది ఒక్క `#settled !== null` తనిఖీ. ఈ ఆలోచనని స్పష్టంగా చెప్పడం interview lo చాలా విలువైనది.

Ravi 10వ తేదీ due • grace 3 రోజులు • గరిష్ఠం ₹450 (పుస్తకం ధర)

తిరిగి ఇచ్చే ముందు - జరిమానా పెరుగుతోంది:

2026-01-12 → ₹0
2026-01-15 → ₹10
2026-01-20 → ₹35

15వ తేదీన తిరిగి ఇచ్చాడు: ₹10

ఆ తర్వాత ఎప్పుడు చూసినా:

2026-01-20 → ₹10 ← మారలేదు ✓
2026-02-09 → ₹10 ← మారలేదు ✓
2026-07-19 → ₹10 ← మారలేదు ✓

ఎప్పటికీ తిరిగి రాని పుస్తకం (cap ₹450):

1 సంవత్సరాల తర్వాత → ₹450
3 సంవత్సరాల తర్వాత → ₹450
10 సంవత్సరాల తర్వాత → ₹450

₹950 → ₹10 (5 రోజులు - 3 grace = 2 రోజులు × ₹5), మరియు ₹18,250 → ₹450.

Part 4 – మూడో విరుపు: నిరీక్షణ జాబితా

9. Step – "reserve" అంటే పేరు రాసుకోవడం

అన్ని ప్రతులూ బయట ఉంటే, సభ్యులు పేరు రాసుకునేవచ్చు:

```
class Title {
  constructor(name) { this.name = name; this.onShelf = 0; this.holds = []; }
  reserve(member, at) { this.holds.push({ member, at }); } // పేరు రాసుకోవడం
  returnCopy() { this.onShelf++; }
  borrow(member) {
    if (this.onShelf === 0) return { ok: false, reason: 'NONE_AVAILABLE' };
    this.onShelf--;
    return { ok: true, to: member };
  }
}
```

ముగ్గురు ఎదురుచూస్తున్నారు:

నిరీక్షణ జాబితా:

ravi	2026-01-01	నుంచి
meera	2026-01-03	నుంచి
kiran	2026-01-05	నుంచి

జాబితా ఉంది, తేదీలు ఉన్నాయి, క్రమం ఉంది. **అంతా బాగుంది** – అనిపిస్తుంది.

10. మూడో విరుపు – 24 రోజులు ఎదురుచూసిన ravi ఓడిపోయాడు

25వ తేదీన ఒక ప్రతి తిరిగి వచ్చింది.
అదే క్షణంలో asha (ఎవరికీ తెలియకుండా browse చేస్తూ) borrow నొక్కింది:
{ ok: true, to: 'asha' }

ravi ప్రయత్నిస్తే : { ok: false, reason: 'NONE_AVAILABLE' }
నిరీక్షణ జాబితా : ravi, meera, kiran ← ఇంకా అలాగే ఉంది

మూడో విరుపు - జాబితా ఉంది, కానీ దాన్ని ఎవరూ చదవరు

Ravi **24 రోజులు** ఎదురుచూశాడు. Asha యాదృచ్ఛికంగా ఆ క్షణంలో app తెరిచింది. **Asha గెలిచింది.**

మరియు ముగ్గురి పేర్లూ ఇంకా జాబితాలోనే ఉన్నాయి - ఎందుకంటే **borrow** ఆ జాబితాని **అసలు చూడనే లేదు.**

మౌలిక తప్పు: **reserve** ఒక **డేటా ఎంట్రీ** మాత్రమే అయింది; అది ఏ ప్రవర్తననీ మార్చలేదు. ఒక జాబితాని నిర్వహించడం మరియు దాన్ని **గౌరవించడం** - రెండూ వేర్వేరు పనులు, మరియు రెండోది ఎవరూ రాయలేదు.

ఇది ఒక సాంకేతిక bug కాదు - **ఇది ఒక వాగ్దాన ఉల్లంఘన.** User కి "మీ స్థానం 1" అని చూపించాం. ఆ సంఖ్యకి అర్థమే లేదు.

11. Step - ప్రతి తిరిగొచ్చినప్పుడు కేటాయించడం

అసలు మార్పు: ఒక ప్రతి తిరిగి వచ్చినప్పుడు, అది **అరలోకి వెళ్ళు** - అది **జాబితాలోని మొదటి వ్యక్తికి కేటాయించబడుతుంది**, ఒక **తీసుకునే గడువుతో**.

```
#allocate(t, now) {
  this.#expireHolds(t, now);
  let free = t.available - t.reservedCount;
  for (const h of t.holds) {
    if (free <= 0) break;
    if (h.allocated) continue;
    h.allocated = true;
    h.expiresAt = now + HOLD_PICKUP_DAYS * DAY;    // తీసుకోవడానికి గడువు
    free--;
  }
}
```

మరియు **borrow** ఇప్పుడు **రెండు ప్రశ్నలు** అడుగుతుంది - "ప్రతి ఉందా?" మరియు "అది **నాకు** ఖాళీయేనా?":

```
const mine = t.holds.find(h => h.member === member);
if (mine && mine.allocated) {
  t.holds.splice(t.holds.indexOf(mine), 1);    // నా hold వాడేశాను
} else if (t.freeToLend <= 0) {
  const head = t.holds.find(h => h.allocated);
  return { ok: false,
    reason: t.available === 0 ? 'NONE_AVAILABLE'
      : `RESERVED_FOR: ${head.member}`,
    waiting: t.holds.length };
}
```

AVAILABLE మరియు **FREETOLEND** – రెండు వేర్వేరు సంఖ్యలు

available = అరలో ఎన్ని ప్రతులు ఉన్నాయి.

freeToLend = అందులో ఎవరికీ వాగ్దానం చేయబడనివి ఎన్ని.

అరలో ఒక ప్రతి **ఉండొచ్చు**, కానీ అది **ravi కోసం పక్కన పెట్టినది** కావచ్చు. ఆ తేడా తెలియకపోతే \$10 విరుపు వస్తుంది.

మరియు UI కి ఆ తేడా **చెప్పాలి**: "అందుబాటులో లేదు" మరియు "ravi కోసం పక్కన పెట్టాం" – రెండూ వేర్వేరు సందేశాలు. అందుకే తిరస్కరణలో **RESERVED_FOR** ఉంది.

మరియు గడువు మీరితే?

Ravi కి కేటాయించాం, కానీ అతను 3 రోజుల్లో రాలేదు. పుస్తకం **ఎప్పటికీ పక్కన ఉండకూడదు**:

```
// గడువులోగా తీసుకోకపోతే – hold విడుదల, ఆ వ్యక్తి జాబితా చివరికి
#expireHolds(t, now) {
  const lapsed = t.holds.filter(h => h.allocated && now >= h.expiresAt);
  for (const h of lapsed) {
    h.allocated = false; h.expiresAt = null; h.at = now;
    t.holds.splice(t.holds.indexOf(h), 1);
    t.holds.push(h); // చివరికి
  }
}
```

జాబితా నుంచి తీసేయట్లేదు – చివరికి జరుపుతున్నాం. ఇది ఒక న్యాయ నిర్ణయం: అతను తన వంతు పోగొట్టుకున్నాడు, కానీ ఆసక్తి పోగొట్టుకోలేదు.

Part 5 – పూర్తి system

12. Step – పునరుద్ధరణ, పరిమితులు, మరియు గడువు మీరిన hold

మూడు చిన్న నియమాలు మిగిలాయి, మరియు మూడూ ఒకదానికొకటి ముడిపడి ఉన్నాయి:

```
renew(loanId, at) {  
  const loan = this.loans.get(loanId);  
  if (!loan || !loan.isOpen) return { ok: false, reason: 'NO_OPEN_LOAN' };  
  const t = loan.copy.title;  
  if (t.holds.length) // వేరేవాళ్ళు ఎదురుచూస్తున్నారు  
    return { ok: false, reason: `OTHERS_WAITING: ${t.holds.length}` };  
  if (loan.renewals >= MAX_RENEWALS)  
    return { ok: false, reason: `MAX_RENEWALS: ${MAX_RENEWALS}` };  
  loan.renewals++;  
  loan.dueAt = Math.max(loan.dueAt, at) + LOAN_DAYS * DAY;  
  return { ok: true, due: iso(loan.dueAt), renewals: loan.renewals };  
}
```

ఆ మొదటి తనిఖీ లేకపోతే \$10 విరుపు వేరే రూపంలో తిరిగి వస్తుంది – Asha పుస్తకాన్ని ఎప్పటికీ renew చేసుకుంటూ ఉంటే, ravi జాబితాలో శాశ్వతంగా ఉండిపోతాడు. నిరీక్షణ జాబితా ఉన్నప్పుడు renew నిరాకరించడం, ఆ జాబితా కదులుతుందని హామీ ఇస్తుంది.

మరియు `Math.max(loan.dueAt, at)` – renew ఎప్పుడు చేసినా గడువు నుంచి 14 రోజులు, ఇవాళ్టి నుంచి కాదు. లేకపోతే ఆలస్యంగా renew చేయడం ఒక బహుమతి అవుతుంది.

13. మొత్తం code ఒకే చోట • నడిపి చూద్దాం

నిర్మాణం • నాలుగు భావనలు

ప్రతిదీ ఒక విరుపుకే జవాబు



```

'use strict';
const DAY = 86400000;
const iso = (t) => new Date(t).toISOString().slice(0, 10);

class Title {
  constructor(isbn, name, author, price = 0) {
    Object.assign(this, { isbn, name, author, price });
    this.copies = [];
    this.holds = []; // [{ member, at, allocated, expiresAt }]
  }
  get total() { return this.copies.length; }
  get onShelf() { return this.copies.filter(c => c.state === 'ON_SHELF'); }
  get available() { return this.onShelf.length; }
  get reservedCount() { return this.holds.filter(h => h.allocated).length; }
  get freeToLend() { return Math.max(0, this.available - this.reservedCount); }
}

class Copy {
  #state = 'ON_SHELF'; // ON_SHELF · LOANED · LOST · REPAIR
  constructor(barcode, title) { this.barcode = barcode; this.title = title; }
  get state() { return this.#state; }
  setState(s) { this.#state = s; }
}

class FinePolicy {
  constructor({ perDay = 5, graceDays = 3, capFraction = 1 } = {}) {
    Object.assign(this, { perDay, graceDays, capFraction });
  }
  amount(dueAt, at, price) {
    const late = Math.floor((at - dueAt) / DAY) - this.graceDays;
    return Math.min(price * this.capFraction, Math.max(0, late) * this.perDay);
  }
}

class Loan {
  #settled = null;
  constructor(id, copy, member, borrowedAt, dueAt, policy) {
    Object.assign(this, { id, copy, member, borrowedAt, dueAt, policy });
    this.returnedAt = null;
    this.renewals = 0;
  }
  fine(now) {
    if (this.#settled !== null) return this.#settled; // గడ్డకట్టినది
    return this.policy.amount(this.dueAt, now, this.copy.title.price);
  }
  settle(at) {
    this.returnedAt = at;
    this.#settled = this.policy.amount(this.dueAt, at, this.copy.title.price);
    return this.#settled;
  }
  get isOpen() { return this.returnedAt === null; }
}

const LOAN_DAYS = 14, HOLD_PICKUP_DAYS = 3, MAX_RENEWALS = 2, MAX_BOOKS = 3;

class Library {
  #seq = 0; #loanSeq = 0;
  constructor({ policy = new FinePolicy() } = {}) {

```

```

    this.titles = new Map();
    this.loans = new Map();
    this.policy = policy;
  }
  #openLoansOf(member) {
    return [...this.loans.values()].filter(l => l.isOpen && l.member === member);
  }

  addTitle(isbn, name, author, price) {
    if (!this.titles.has(isbn))
      this.titles.set(isbn, new Title(isbn, name, author, price));
    return this.titles.get(isbn);
  }
  addCopy(isbn) {
    const t = this.titles.get(isbn);
    if (!t) return { ok: false, reason: `NO_TITLE: ${isbn}` };
    const c = new Copy(`${isbn}-c${++this.#seq}`, t);
    t.copies.push(c);
    return { ok: true, barcode: c.barcode };
  }
  search(q) {
    return [...this.titles.values()]
      .filter(t => t.name.toLowerCase().includes(q.toLowerCase()))
      .map(t => ({ name: t.name, total: t.total,
        available: t.available, waiting: t.holds.length }));
  }

  reserve(isbn, member, at) {
    const t = this.titles.get(isbn);
    if (!t) return { ok: false, reason: `NO_TITLE: ${isbn}` };
    if (t.holds.some(h => h.member === member))
      return { ok: false, reason: 'ALREADY_RESERVED' };
    t.holds.push({ member, at, allocated: false, expiresAt: null });
    this.#allocate(t, at);
    return { ok: true, position: t.holds.findIndex(h => h.member === member) + 1 };
  }

  #allocate(t, now) {
    this.#expireHolds(t, now);
    let free = t.available - t.reservedCount;
    for (const h of t.holds) {
      if (free <= 0) break;
      if (h.allocated) continue;
      h.allocated = true;
      h.expiresAt = now + HOLD_PICKUP_DAYS * DAY; // తీసుకోవడానికి గడువు
      free--;
    }
  }
  // గడువులోగా తీసుకోకపోతే - hold విడుదల, ఆ వ్యక్తి జాబితా చివరికి
  #expireHolds(t, now) {
    const lapsed = t.holds.filter(h => h.allocated && now >= h.expiresAt);
    for (const h of lapsed) {
      h.allocated = false; h.expiresAt = null; h.at = now;
      t.holds.splice(t.holds.indexOf(h), 1);
      t.holds.push(h); // చివరికి
    }
  }

  borrow(isbn, member, at) {

```

```

const t = this.titles.get(isbn);
if (!t) return { ok: false, reason: `NO_TITLE: ${isbn}` };
if (this.#openLoansOf(member).length >= MAX_BOOKS)
  return { ok: false, reason: `LIMIT_REACHED: ${MAX_BOOKS}` };
this.#allocate(t, at);

const mine = t.holds.find(h => h.member === member);
if (mine && mine.allocated) {
  t.holds.splice(t.holds.indexOf(mine), 1); // నా hold వాడేశాను
} else if (t.freeToLend <= 0) {
  const head = t.holds.find(h => h.allocated);
  return { ok: false,
    reason: t.available === 0 ? 'NONE_AVAILABLE'
      : `RESERVED_FOR: ${head.member}`,
    waiting: t.holds.length };
}
const copy = t.onShelf[0];
copy.setState('LOANED');
const loan = new Loan(`L${++this.#loanSeq}`, copy, member, at,
  at + LOAN_DAYS * DAY, this.policy);
this.loans.set(loan.id, loan);
return { ok: true, loan: loan.id, due: iso(loan.dueAt) };
}

return_(loanId, at) {
  const loan = this.loans.get(loanId);
  if (!loan || !loan.isOpen) return { ok: false, reason: 'NO_OPEN_LOAN' };
  const fine = loan.settle(at);
  loan.copy.setState('ON_SHELF');
  const t = loan.copy.title;
  this.#allocate(t, at); // జాబితాలో తర్వాతివారికి
  const next = t.holds.find(h => h.allocated);
  return { ok: true, fine, nextInline: next?.member ?? null };
}

renew(loanId, at) {
  const loan = this.loans.get(loanId);
  if (!loan || !loan.isOpen) return { ok: false, reason: 'NO_OPEN_LOAN' };
  const t = loan.copy.title;
  if (t.holds.length)
    return { ok: false, reason: `OTHERS_WAITING: ${t.holds.length}` };
  if (loan.renewals >= MAX_RENEWALS)
    return { ok: false, reason: `MAX_RENEWALS: ${MAX_RENEWALS}` };
  loan.renewals++;
  loan.dueAt = Math.max(loan.dueAt, at) + LOAN_DAYS * DAY;
  return { ok: true, due: iso(loan.dueAt), renewals: loan.renewals };
}

duesOf(member, now) {
  return this.#openLoansOf(member)
    .map(l => ({ loan: l.id, book: l.copy.title.name,
      due: iso(l.dueAt), fine: l.fine(now) }));
}
}

```

--- రెండు ప్రతులు బయటికి ---

```
asha : { ok: true, loan: 'L1', due: '2026-01-15' }
```

```

sai : { ok: true, loan: 'L2', due: '2026-01-16' }
kiran: { ok: false, reason: 'NONE_AVAILABLE', waiting: 0 }

--- నిరీక్షణ జాబితా ---
ravi reserve : { ok: true, position: 1 }
meera reserve : { ok: true, position: 2 }
ravi మళ్ళీ : { ok: false, reason: 'ALREADY_RESERVED' }

--- asha renew చేయాలనుకుంటే ---
{ ok: false, reason: 'OTHERS_WAITING: 2' }

--- asha 20వ తేదీన తిరిగి ఇచ్చింది (due 15) ---
{ ok: true, fine: 10, nextInLine: 'ravi' }
ఇప్పుడు kiran ప్రయత్నిస్తే : { ok: false, reason: 'RESERVED_FOR: ravi', waiting: 2 }
ravi ప్రయత్నిస్తే : { ok: true, loan: 'L3', due: '2026-02-03' }

--- ravi గడువులోగా రాకపోతే (3 రోజులు) ---
sai తిరిగి ఇచ్చాడు : { ok: true, fine: 0, nextInLine: 'ravi' }
ravi 15వ తేదీన వస్తే : { ok: false, reason: 'RESERVED_FOR: meera', waiting: 2 }
meera 15వ తేదీన : { ok: true, loan: 'L2', due: '2026-01-29' }

--- పుస్తకాల పరిమితి ---
4వ పుస్తకం : { ok: false, reason: 'LIMIT_REACHED: 3' }

```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

kiran: NONE_AVAILABLE, waiting: 0 – ప్రతులు లేవు, మరియు జాబితా ఖాళీ. అతను reserve చేసుకోవచ్చుని UI చెప్పగలదు.

OTHERS_WAITING: 2 – asha renew చేయలేదు. §12 lo చెప్పిన నియమం. మరియు కారణం **ఎంత మంది ఎదురుచూస్తున్నారో** చెప్తోంది.

fine: 10, nextInLine: 'ravi' – 5 రోజులు ఆలస్యం – 3 grace = $2 \times ₹5 = ₹10$. మరియు తిరిగి ఇచ్చిన ప్రతిస్పందనలోనే **తర్వాతివారి పేరు** ఉంది – librarian కి అదే కావాలి.

kiran: RESERVED_FOR: ravi – **§10 ఏరుపు సరిచేయబడింది**. ప్రతి అరలో ఉంది, కానీ kiran కి కాదు. మరియు తిరస్కరణ **NONE_AVAILABLE** కాదు – **ఎందుకో** చెప్తోంది.

ravi 15వ తేదీన వస్తే: RESERVED_FOR: meera – ఇది సూక్ష్మమైనది. Ravi కి 10వ తేదీన కేటాయించాం, గడువు 13. అతను 15న వచ్చాడు. అతని hold **గడువు మీరింది**, అతను జాబితా చివరికి వెళ్ళాడు, మరియు ప్రతి **meera** కి వెళ్ళింది. మరియు ravi కి ఆ కారణం కనిపిస్తోంది.

L2 రెండుసార్లు కనిపించింది – ఎందుకంటే ఇవి **రెండు వేర్వేరు Library వస్తువులు**, ఒక్కొక్కటి తన సొంత **#loanSeq** తో. Ids ఒక వస్తువు లోపల ప్రత్యేకం, globally కాదు – **అది ఒక design ఎంపిక**, మరియు నిజమైన systems lo ఇది ఒక UUID అవుతుంది.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	<code>Book</code> , <code>Map<isbn, Book></code>	మౌలిక అస్థిపంజరం
§4 (ఏరుపు)	<code>Title</code> + <code>Copy</code> విడిగా	ప్రతి కొంటే loan మాయమైంది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§5	<code>available</code> ఒక getter	నిల్వ చేసిన లెక్క పాతబడుతుంది
§5	<code>Copy.state</code>	<code>LOST</code> మరియు <code>REPAIR</code> వేర్వేరు పరిస్థితులు
§7 (విరుపు)	<code>#settled</code>	తిరిగి ఇచ్చాక జరిమానా పెరిగింది
§8	<code>FinePolicy</code> (grace, cap)	₹450 పుస్తకానికి ₹18,250
§10 (విరుపు)	<code>#allocate</code> , <code>freeToLend</code>	24 రోజులు ఎదురుచూసినవాడు ఓడిపోయాడు
§11	<code>#expireHolds</code>	పుస్తకం ఎప్పటికీ పక్కన ఉండకూడదు
§12	<code>OTHERS_WAITING</code>	Renew జాబితాని ఆపేస్తుంది

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Code సులభం – సమయం అంతా నియమాల మీదే

6 నిమి	Clarify – పలు ప్రశ్నలూ? జరిమానా? reserve?
10 నిమి	Title vs Copy · ఇదే పునాది
9 నిమి – Fine	గడ్డకట్టడం · grace · cap
13 నిమి – Holds	కేటాయింపు · గడువు · renew నిరాకరణ
7 నిమి	పరిమితులు · శాఖలు · notifications

ఏమి తప్పక చెప్పాలి

1. Title మరియు Copy వేర్వేరు (§5) – మొదటి 3 నిమిషాల్లో. ఇది లేకుండా మిగతాదంతా తప్పు పునాది మీద ఉంటుంది.
2. Fine తిరిగి ఇచ్చిన క్షణంలో గడ్డకట్టాలి (§8) – "otherwise a returned book keeps accruing." ఇది గుర్తుండిపోయే వాక్యం.
3. Fine కి ఒక cap (§8) – "never more than the replacement cost." ఇది product ఆలోచనని చూపిస్తుంది.
4. Hold ని గౌరవించడం, కేవలం నమోదు చేయడం కాదు (§11) – available vs freeToLend .
5. Holds ఉంటే renew నిరాకరణ (§12) – లేకపోతే జాబితా ఎప్పటికీ కదలదు.

ఏమి వదిలేయాలి

- search ని లోతుగా వెళ్ళొద్దు – "linear scan for now; a real one needs an inverted index" అని చెప్పి ముందుకి. (Deep Dive 17 Io అది వస్తుంది.)
- Member classes, ఖాతా వివరాలు – ఒక memberId చాలు.
- #expireHolds రాయొద్దు – "holds have a pickup window; if it lapses they go to the back" అని ఒక వాక్యం.
- Database schema – అడిగితేనే.

15. నీటితో చెప్పాల్సిన English script

"Three questions before I draw anything. Does the library hold multiple copies of the same book? Are there late fees – and is there a grace period or a cap? And can members reserve a book that's currently out?

The first one decides the whole model. The obvious design is one Book class keyed by ISBN, and it works until the library buys a second copy. Then the new copy overwrites the old record, and whoever had the first one borrowed silently disappears from the system – no reminder, no fine, and now someone else can borrow a book that's already out.

So I'd split it in two. A Title is the catalogue entry – ISBN, name, author, price. A Copy is a physical object with its own barcode and its own state: on shelf, loaned, lost, in repair. Search returns Titles with a count; borrowing takes a specific Copy. And availability is a computed property over the copies, not a stored flag, because a stored count goes stale the moment anything else touches a copy.

For fines, the natural formula is days late times a rate, evaluated now. That's right while the book is still out and wrong the moment it comes back – a book returned five days late in January still shows a growing fine in July. So the loan freezes the fine when it's settled: if it's been returned, return the frozen value; otherwise compute it live. Same method, two meanings, and the distinction is the whole point.

I'd also put the fine rule behind a policy object with a per-day rate, a grace period and a cap, because those vary by library. The cap especially – without it a four-hundred-and-fifty-rupee book accrues eighteen thousand rupees over ten years, which isn't a fine any more, it's just a wrong number. The cap is the replacement cost.

On reservations – this is the one people get wrong in a way that matters. It's easy to write reserve as appending a name to a list, and then the list does nothing: when a copy comes back, whoever happens to be browsing gets it, and the person who waited three weeks is still sitting at position one. So the return path has to allocate: the returning copy is assigned to the head of the queue with a pickup window, and borrow has to distinguish between copies that are on the shelf and copies that are on the shelf and unpromised. If the pickup window lapses, that hold is released and the member moves to the back of the queue rather than being dropped.

One more rule that ties into it: I'd refuse renewal when anyone is waiting. Otherwise the current borrower can renew forever and the queue never moves, which quietly recreates the same unfairness. And renewal should extend from the due date, not from today, so renewing late isn't a reward."

16. Follow-ups – బహుళ శాఖలు, e-books, notifications

Follow-up	జవాబు	మారే classes
"ఒక సభ్యుడు పుస్తకం పోగొట్టుకున్నాడు"	<code>Copy.setState('LOST')</code> + జరిమానా = cap	0 కొత్తవి
"వేర్వేరు సభ్యత్వాలు – విద్యార్థి, అధ్యాపకుడు"	<code>MembershipPolicy</code> – loan రేజులు, పరిమితి, జరిమానా రేటు	+1 కొత్తది
"పుస్తకం సిద్ధమైనప్పుడు తెలియజేయాలి"	Deep Dive 07 event bus – <code>HOLD_READY</code> publish	0 concepts
"బహుళ శాఖలు"	<code>Copy</code> కి ఒక <code>branch</code> ; holds శాఖ వారీగా లేదా ఉమ్మడిగా	<code>Copy</code> , <code>#allocate</code>
"E-books – అపరిమిత ప్రతులు"	<code>Title</code> కి <code>isDigital</code> ; ప్రతులు అవసరం లేదు, కానీ <code>licence</code> పరిమితి ఉండొచ్చు	<code>Title</code>
"Search నెమ్మదిగా ఉంది"	Deep Dive 17 – inverted index / trie	Search layer
"ఇద్దరు librarians ఒకేసారి చివరి ప్రతి"	కింద చూడండి	Storage layer

ఒకేసారి – ఆ పాత శత్రువు

"నా `borrow` lo `t.onShelf[0]` ని తీసుకుని `setState('LOANED')` చేస్తున్నాను. ఇద్దరు librarians ఒకే క్షణంలో దాన్ని చేస్తే – ఇద్దరికీ అదే ప్రతి కనిపిస్తుంది.

Deep Dive 04 §7 lo ఇదే సమస్యని చూశాం. పరిష్కారం అక్కడిదే: **ఆ ప్రతి స్థితిని మార్చడం అణుసంబంధమైనది (atomic) కావాలి** – database lo ఒక షరతుతో కూడిన update (`UPDATE copies SET state='LOANED' WHERE barcode=? AND state='ON_SHELF'`), మరియు అది 0 rows ప్రభావితం చేస్తే ఇంకొకరు ముందే తీసుకున్నారని అర్థం.

మరియు ఇక్కడ ఒక అదనపు చిక్కు ఉంది: `#allocate` కూడా holds ని మార్చుతుంది. Return మరియు `borrow` ఒకేసారి జరిగితే, ఒక ప్రతి రెండుసార్లు కేటాయించబడొచ్చు. కాబట్టి **title స్థాయిలో ఒక lock** అవసరం – ఒక్కో పుస్తకానికి ఒక్కటి, మొత్తం గ్రంథాలయానికి కాదు."

17. ఏమి నేర్చుకున్నాం

అలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
"ఇది ఏమిటి" vs "ఇది ఏది"	Title vs Copy (§5)	Deep Dive 04 · SKU vs item, model vs instance
ఉత్పన్న విలువని లెక్కించు, నిల్వ చేయొద్దు	<code>available</code> getter (§5)	Deep Dive 05, 11, 12

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
గత సంఘటన మారకూడదు	<code>#settled</code> (§8)	Deep Dive 11 §5 · invoices, payslips
ప్రతి నియమానికీ ఒక హద్దు	Fine cap (§8)	Retries, timeouts, ర్యాంకులు
జాబితాని నమోదు చేయడం ≠ గౌరవించడం	<code>#allocate</code> (§11)	Queues, priorities, SLAs
రెండు సంఖ్యలు, రెండు అర్థాలు	<code>available</code> vs <code>freeToLend</code> (§11)	Inventory, capacity planning
తిరస్కరణ కారణం చెప్పాలి	<code>RESERVED_FOR</code> (§13)	Deep Dive 03 §12, 11 §11, 14 §13

ఒక చివరి ఆలోచన – "సులభమైన" **PROBLEM** అంటే ఏమిటి

ఈ problem ని చాలా మంది తక్కువగా అంచనా వేస్తారు, మరియు interviewers అందుకే దీన్ని ఎంచుకుంటారు.

ఇక్కడ కష్టమైన algorithm ఒక్కటే లేదు. Data structure ఒక Map మరియు ఒక array. కానీ మూడు విరుపులూ "తప్పు" లాగా కనిపించవు – అవి అసంపూర్ణమైన ఆలోచన లాగా కనిపిస్తాయి.

మరియు అదే interviewer చూసేది: మీరు code రాయగలరా అని కాదు – నిజమైన గ్రంథాలయంలో ఏమి జరుగుతుందో మీరు ఊహించగలరా అని. రెండో ప్రతి. ఆరు నెలల తర్వాత. ఎదురుచూస్తున్న వ్యక్తి.

LLD interview అనేది ఒక coding పరీక్ష కాదు – ఇది ఒక ఊహాశక్తి పరీక్ష.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery · 12 File System · 13 Leaderboard · 14 Text Editor · 15 Library.**

వేగవంతమైన revision కోసం – `LLD_Design_Problems_Telugu.pdf` .

Library Management – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output నిజంగా `node` lo run చేసి తీసినదే ✓